React

☑ Also known as React.js/ React.JS

Free & Open-Source JavaScript Library for building user interfaces (UIs).

React is **Created by Meta (Facebook)** & is now maintained by Meta & community of developers.

- HTML/CSS/JS code refreshes the whole page unexpectedly upon interaction (like clicking a button, submitting a form, etc.)

React Features:

- React is used to build *SPAs (Single Page Applications)*
- Creates *reusable UI components*
- Develops dynamic, fast & scalable front end applications.
- ReactJS uses a Virtual DOM mechanism to efficiently update the HTML DOM.
- Faster Navigation (No full page reloads)
- Allows writing HTML-like syntax inside JavaScript through JSX.
- Data flows in a single direction (parent → child).
- With React Native, the same concepts can be used to build mobile apps for Android and iOS.
- Huge ecosystem of libraries, tools, and community support.

👉 ReactJS uses a *Virtual DOM* mechanism to efficiently update the *HTML DOM*.

Instead of reloading the entire DOM, the virtual DOM updates only the specific elements that have changed, resulting in faster performance.

🥷 React follows a *Component-Driven Architecture*

The user interface is built by composing independent, reusable pieces called components. Each component manages its own logic and rendering, making development more modular, scalable, and easier to maintain.

💡 **A component is a reusable piece of code that represents a specific part of the user interface (UI).**

React Element : An object that describes the properties of an actual DOM node which will be created by React.

It contains information (like `type`, `props`, and `children`) about the actual DOM node that React will create and manage.

Simple Structure of a React Elements:

```
{
  type: "button",
  props: {
```

```
    className: "btn",
    children: "Click Me"
  }
}
```

📝 Explanation:

- `type` → The type of DOM element (e.g., "div", "button", etc.)
- `props` → The attributes or properties passed to the element
- `children` → The inner content or nested elements (optional)

📃 SPAs(Single Page Application)

A web application that loads **a single HTML page** and dynamically updates content as users interact with it is known as a Single Page Application (SPA).

In an SPA, the entire page doesn't reload with every interaction; only the necessary parts of the page are updated, providing a smoother and faster user experience.

- It uses client side rendering (CSR) [Javascript Updates the DOM]
- Communicates to server via APIs (e.g. REST, GraphQL etc.)

Example: Facebook, Gmail, Twitter

☑️ Pros

- Faster Navigation (No full page reloads)
- Smoother UX
- Easier to build complex, interactive UIs
- Good for real time updates

❌ Cons

- Slower initial loads (Heavy JS Bundles)
- Search engines historically struggled with Javascript heavy SPAs. But solutions like Next.JS do help.
- Browser history management needs extra efforts (e.g React Router)

🐞 MPAs(Multi Page Application)

A traditional web app where each page is a seperate HTML document loaded from the server.

- It uses Server side rendering (SSR) [Server generats HTML for each request]

Example: Amazon, Wordpress, Old school sites

☑️ Pros

- Better SEO out of the box as each page is a seperate HTML page.
- faster initial loads (Less Javascript code, server handles rendering)

- Easier to scale as pages can be cached independently.

❌ Cons

- Slower navigation (full page reloads upon each interaction)
- Less interactivity (more clunky UX)
- Harder to maintain state (e.g Shopping Cart across pages)